

Trigger Interview Questions with Exact Blueprint

1.Explain Trigger Framework you worked on?

1. Trigger Handler Pattern
2. Trigger Framework using a Virtual Class
3. Trigger Framework using an Interface
4. An architecture framework to handle triggers

I have worked on Trigger Handler Pattern. The basic idea behind the Trigger Handler pattern is to separate the trigger logic into a separate class from the trigger itself. The trigger class is responsible for registering the trigger with the appropriate events and for delegating the actual logic to the handler class.

2. How to avoid recursion in trigger?

1. Use Static Boolean Variable
2. Use Static **Set** to Store Record Id.
3. Use Static **Map**
4. Use Old Map
5. Follow Best practices for triggers

3.What is the best practices for trigger in Salesforce?

- 1) One Trigger Per Object
- 2) Logic-less Triggers
- 3) Context-Specific Handler Methods

- 4) Bulkify your Code
- 5) Avoid SOQL Queries or DML statements inside FOR Loops
- 6) Using Collections and Efficient For Loops
- 7) Querying Large Data Sets
- 8) Use @future Appropriately
- 9) Avoid Hardcoding IDs

4. What are Types of Triggers?

- Before Triggers: – Before triggers are used to perform the logic on the same object and specifically we cannot use the DML operation (Insert, update, delete) on these triggers. These triggers fired before the data saved into the database.
- After Triggers: – After triggers are used to perform the logic on the related objects and these are fired when the record gets saved to the database but not committed.

5.What are Trigger Events?

Trigger Events: – Trigger events are the database activities that cause a trigger to run.

Insert, Update, Delete, Undelete

Therefore, in conjunction with the types of triggers we can have the following below Trigger Events.

Trigger Events
Before Insert
After Insert
Before Update
After Update

Before Delete
After Delete
After Undelete

6.How do you use context variables in a Salesforce trigger?

Context variables in a Salesforce trigger provide information about the state of the records

- **Trigger.new:** A list of the new versions of the records being inserted or updated.
- **Trigger.old:** A list of the old versions of the records being updated or deleted.
- **Trigger.newMap:** A map of IDs to the new versions of the records.
- **Trigger.oldMap:** A map of IDs to the old versions of the records.
- **Trigger.isInsert, Trigger.isUpdate, Trigger.isDelete, and Trigger.isUndelete:** Boolean values that indicate the type of DML operation.

7.What is the purpose of Trigger.new and Trigger.old in Salesforce?

Trigger.new and Trigger.old are context variables used in Salesforce triggers to access the records involved in a DML operation:

- **Trigger.new:** Contains a list of the new versions of the records that are being inserted or updated. In an insert operation, Trigger.new contains the records being inserted. In an update operation, it contains the new values of the records after the update.
- **Trigger.old:** Contains a list of the old versions of the records that are being updated or deleted. It is not available in insert operations. Trigger.old is useful for comparing changes between the old and new

values during an update operation or for knowing the state of records before deletion.

8.How to do a simple update on the Description field of the Lead Object, when a new Lead is Created or Updated, this Trigger will run.

```
for(Lead ld : Trigger.New)
{
// This will Update the Lead Description Field on before Insert and before
Update Event.
ld.Description = 'This is an Update from LeadDemoTrigger';
}

=====

trigger LeadDemoTrigger on Lead (before insert , before Update) {
    for(Lead ld : Trigger.New)  For Loop on Lead records iterating over records present in Trigger.New Context
    Variable
    {
        // This will Update the Lead Description Field on before Insert and before Update Event.
        ld.Description = 'This is an Update from LeadDemoTrigger';
    }
}  
loop variable API name of the Lead Description field
```

Code Explanation line by line

- The **for each** loop will Iterate through the records that initiated the Trigger and that record can be accessed using the Trigger.New context variable, so you don't have to worry about if the action is been performed on the correct record or not.
- The for each loop will run as many number of times as there are records in the Trigger.new context variable list.
- As I have already stated that Before triggers are used for performing actions on the same object records. The Lead record initiated the Trigger and the same record 'Description' field is updated in the run.

- Lastly and the most important thing that you can notice we just mentioned the line, it's the main logic of Trigger. (we can reference the fields of a record using the dot(.) notation)

Id.Description = 'This is an Update from LeadDemoTrigger';

without any DML statement to enforce the change, please note and always keep in mind, we never need DML statements to do any update/changes in record's field in case of Before Triggers.

9.Scenario: – Create a follow up Task on the Opportunity record when the Opportunity stage is set to 'Closed Won'.

In the context of your scenario (creating a follow-up Task when an Opportunity stage is set to 'Closed Won'):

- **After Trigger:** An "after" trigger is appropriate because you want to perform actions (like creating a Task) that depend on the Opportunity record being fully saved and available in the database, including its ID and related field values.

General Blueprint for After Update Trigger on Parent Object

(Insert New Child Records)

1. **Store Parent Record IDs:** Collect the IDs of parent records that meet specific conditions and for which new child records need to be created.
2. **Set up the list and prepare to add new records to it.**
3. **For Loop to Create New Child Records:** Iterate through the list of parent IDs to create new child records for each relevant parent.
4. **Add to List:** Add each newly created record to the list.
5. **DML Operation:** Perform a DML operation (insert) on the list of new child records.

```
trigger OpportunityAfterUpdateTrigger on Opportunity (after update) {
```

```
// 1. Store Parent Record IDs
```

```
List<Id> oppIds = new List<Id>();  
for (Opportunity opp : Trigger.new) {  
    if (opp.StageName == 'Closed Won' &&  
Trigger.oldMap.get(opp.Id).StageName != 'Closed Won') {  
        oppIds.add(opp.Id);  
    }  
}
```

```
// 2. Prepare for Child Record Creation
```

```
if (!oppIds.isEmpty()) {  
    List<Task> tasksToCreate = new List<Task>(); // Initialize the list for new  
Tasks
```

```
// 3. For Loop to Create New Child Records
```

```
for (Id oppId : oppIds) {  
    tasksToCreate.add(new Task(  
        WhatId = oppId,  
        Subject = 'Follow up with the client',  
        Status = 'Not Started',  
        Priority = 'Normal',  
        ActivityDate = Date.today() + 7  
    ));  
}
```

// 5. DML Operation

```
if (!tasksToCreate.isEmpty()) {  
    insert tasksToCreate;  
}  
}  
}
```

```
1 trigger OpportunityStageChange on Opportunity (after insert , after update) {  
2  
3     List<Task> tskList = new List<Task>();  
4  
5     // Logic to Iterate over all the Opportunities which are Inserted/Updated as closed won.  
6     for(Opportunity opp : Trigger.new)  
7     {  
8         if(opp.StageName == 'Closed Won') // Condition check if the Opportunity in focus has Stage = 'Closed Won'  
9         {  
10            //Task Creation  
11            Task tsk = new Task(WhatId = opp.Id, Subject = 'Follow up Task- Opportunity Closed Won', Priority = 'High');  
12            tskList.add(tsk);  
13        }  
14    }  
15    //Check if the List is not Empty.  
16    if(tskList.size() > 0)  
17    {  
18        insert tskList; // DML Operation to insert the Task Records.  
19    }  
20 }
```

Task
Follow up Task- Opportunity Closed Won

Click to add topics: ?

1 7 31

Back to List: Opportunities

Attachments (0)

Edit Delete Create Follow-Up Task Create Follow-Up Event

Assigned To	Abhishek Chogora	Status	Not Started
Subject	Follow up Task- Opportunity Closed Won	Name	
Due Date		Related To	Express Logistics Portable Truck Generators
Phone		Email	
Priority	High		
Created By	Abhishek Chogora, 5/22/2019 11:54 PM	Last Modified By	Abhishek Chogora, 5/22/2019 11:54 PM
Comments			

API Name of this field is WhatId

10. Scenario to Update Existing Task where task Status is 'Not Stated' and related Opportunity Status is Closed Won. Change Task Status to Completed.

General Blueprint for After Update Trigger on Parent Object

(Update Child Record)

1. **Store Parent Record IDs** - Collect the IDs of the parent records that meet a specific condition or have been updated.
2. **SOQL on Child Records** - Perform a SOQL query to retrieve related child records based on the parent IDs collected which need to be updated.
3. **For Loop to Process Child Records** - Iterate through the list of retrieved child records.
4. **Add to List** - Make any necessary updates or modifications to the child records and add them to a list for DML operations.
5. **DML Operation** - Perform the appropriate DML operation (insert, update, delete) on the list of child records.

trigger OpportunityAfterUpdateTrigger on Opportunity (after update) {

// 1. Store Parent Record IDs – this step remains same

List<Id> oppIds = new List<Id>();

// Loop through updated Opportunity records

for (Opportunity opp : Trigger.new) {

// Check if the Opportunity's stage is set to 'Closed Won' and was not 'Closed Won' before – this step remains same


```
        if (opp.StageName == 'Closed Won' &&  
Trigger.oldMap.get(opp.Id).StageName != 'Closed Won') {  
            oppIds.add(opp.Id); // Store the ID of this Opportunity  
        }  
    }  
}
```

// 2. SOQL on Child Records if there are relevant Opportunity IDs

```
if (!oppIds.isEmpty()) {  
    // Query to get related Task records  
    List<Task> tasksToUpdate = [SELECT Id, Status FROM Task WHERE WhatId  
IN :oppIds AND Status = 'Not Started'];
```

// 3. For Loop to Process Child Records

```
for (Task t : tasksToUpdate) {
```

// 4. Add to List after modifying necessary fields

```
    t.Status = 'Completed'; // Update Task status to 'Completed'  
}
```

// 5. DML Operation to update the records in the database

```
if (!tasksToUpdate.isEmpty()) {  
    update tasksToUpdate;  
}  
}  
}
```

11. Scenario – Create related Contact record whenever a new Account record is created.

First, we recognize that trigger has to be written on Account i.e. Parent Object as when action is taken on Parent i.e. when New Account record is created to something to child. i.e. Based on A something to B.

Second, it will be After Trigger as related object has to be updated.

We'll follow the blueprint steps for inserting new child records based on Parent Record.

Blueprint for After Insert Trigger on Parent Object (Insert New Child Records)

- 1. Store Parent Record IDs:** Collect the IDs of newly created parent records (Accounts).
- 2. Prepare for Child Record Creation** - Initialize a list to hold the new child records (Contacts) to be created.
- 3. For Loop to Create New Child Records:** Iterate through the list of parent IDs to create new child records for each relevant parent.
- 4. Add to List:** Collect the new child records into a list for bulk insertion.
- 5. DML Operation:** Insert the new child records into the database.

```
trigger AccountAfterInsertTrigger on Account (after insert) {
```

// 1. Store Parent Record IDs

```
List<Id> accountIds = new List<Id>();  
for (Account acc : Trigger.new) {  
    accountIds.add(acc.Id); // Store the ID of the new Account  
}
```

// 2. Prepare for Child Record Creation

```
if (!accountIds.isEmpty()) {  
    List<Contact> contactsToCreate = new List<Contact>(); // Initialize the list  
    for new Contacts
```

// 3. For Loop to Create New Child Records

```
for (Id accountId : accountIds) {  
    contactsToCreate.add(new Contact(  
        AccountId = accountId, // Link Contact to the Account  
        LastName = 'Default Last Name', // Set default Last Name (required  
field)  
        Email = 'sfdcamlified@example.com'  
    ));  
}
```

// 5. DML Operation

```
if (!contactsToCreate.isEmpty()) {  
    insert contactsToCreate; // Insert the new Contacts into the database  
}  
}  
}
```

Line by Line Explanation

1. We loop through Trigger.new, which contains the new Account records that have been inserted, and store their IDs in the accountIds list.

2. We initialize a list `contactsToCreate` to hold the new Contact records that we will create.
3. We iterate over the `accountIds` list. For each Account ID, we create a new Contact record. We set necessary fields like `AccountId` to link the Contact to the Account, and `LastName` which is a required field for Contacts.
4. As we create each new Contact record, we add it to the `contactsToCreate` list.
5. Finally, we perform an insert DML operation on the `contactsToCreate` list to save the new Contact records to the database.

12. Update Account Rating to 'Hot' on account when opportunity stage equals 'closed one'

We need to write a trigger on the Opportunity object (the child) that updates the Account (the parent) when the Opportunity's stage is set to 'Closed Won'. The account's rating should be updated to 'Hot'.

Blueprint for After Update Trigger on Child Object (Update Parent Record)

- 1. Store Child Record IDs:** Collect the IDs of child records (Opportunities) that meet a specific condition (`stage = 'Closed Won'`).
- 2. SOQL on Parent Records:** Retrieve the parent records (Accounts) related to the child records stored in step 1.
- 3. For Loop to Process Parent Records:** Iterate through the list of retrieved parent records to identify and make necessary updates.
- 4. Add to List:** Collect the modified parent records into a list for bulk processing.

5. DML Operation: Save the changes made to the parent records.

```
trigger OpportunityAfterUpdate on Opportunity (after update) {  
    // 1. Store Child Record IDs  
    Set<Id> accountIds = new Set<Id>();  
    for (Opportunity opp : Trigger.new) {  
        if (opp.StageName == 'Closed Won' &&  
            Trigger.oldMap.get(opp.Id).StageName != 'Closed Won') {  
            accountIds.add(opp.AccountId); // Store the Account ID related to the  
            Opportunity  
        }  
    }  
}
```

```
    // 2. SOQL on Parent Records  
    if (!accountIds.isEmpty()) {  
        List<Account> accountsToUpdate = [SELECT Id, Rating FROM Account  
        WHERE Id IN :accountIds];
```

```
    // 3. For Loop to Process Parent Records  
    for (Account acc : accountsToUpdate) {  
        acc.Rating = 'Hot'; // Update the Rating field to 'Hot'  
    }  
}
```

```
    // 4. DML Operation to update the records in the database  
    if (!accountsToUpdate.isEmpty()) {
```

```
        update accountsToUpdate;  
    }  
}  
}
```

13. Write a trigger that updates the "City" field in an Account when the same field is updated in an Opportunity.

Blueprint for After Update Trigger on Child Object (Update Parent Record)

- 1. Store Child Record IDs:** Identify and store the IDs of Opportunities where the City field has changed.
- 2. SOQL on Parent Records:** Query the parent Accounts related to these Opportunities.
- 3. For Loop to Process Parent Records:** Update the City field in the related Accounts based on the Opportunity's City.
- 4. DML Operation:** Perform an update operation on the modified Accounts.

Mind Map

Now that our blueprint is ready. Let us make a **mind map** of what we will do.

- 1. Store Child Record IDs and Account IDs to Update:** We use a map (**accountCityMap**) to store Account IDs as keys and the new City values as values. This map helps us efficiently update the related Account's City field.
Note: We are using map as it says we need to update account city same as

opportunity city field so we need to save it apart from the id so that we can utilize it to update account city in later steps.

2. SOQL on Parent Records: **We query the Accounts that need to be updated using the IDs stored in accountCityMap.** This ensures we only fetch the Accounts that are related to Opportunities where the City field has changed.

3. For Loop to Process Parent Records: **We iterate over the fetched Account records and update their BillingCity** with the new City value from the Opportunities.

4. DML Operation: Finally, we **update the modified Account records in the database.**

Now time for Code

```
trigger OpportunityCityUpdateTrigger on Opportunity (after update) {
```

```
    // 1. Store Child Record IDs and Account IDs to update
```

```
    Map<Id, String> accountCityMap = new Map<Id, String>();
```

```
    for (Opportunity opp : Trigger.new) {
```

```
        // Check if the City field has changed
```

```
        if (opp.City__c != Trigger.oldMap.get(opp.Id).City__c) {
```

```
            // Store the Account ID and new City value
```

```
            accountCityMap.put(opp.AccountId, opp.City__c);
```

```
        }
```

```
    }
```

// 2. SOQL on Parent Records

```
if (!accountCityMap.isEmpty()) {  
    List<Account> accountsToUpdate = [SELECT Id, BillingCity FROM Account  
WHERE Id IN :accountCityMap.keySet()];
```

// 3. For Loop to Process Parent Records

```
    for (Account acc : accountsToUpdate) {  
        acc.BillingCity = accountCityMap.get(acc.Id); // Update the City field in  
the Account  
    }
```

// 4. DML Operation to update the records in the database

```
    if (!accountsToUpdate.isEmpty()) {  
        update accountsToUpdate;  
    }  
}  
}
```